

中山大学计算机院本科生实验报告

(2024年春季学期)

课程名称：并行程序设计

批改人：

实验	5-OpenMP矩阵乘法	专业	计算机科学与技术
学号	22336203	姓名	沈苇杭
Email	shenwh7@mail2.sysu.edu.cn	完成日期	2025.4.28

实验目的

- 掌握OpenMP矩阵乘法的API。
- 分析线程数量、矩阵规模对运行时间的影响。
- 了解OpenMP调度方式，分析不同调度方式对运行时间的影响。

实验过程与核心代码

```
int main(int argc, char *argv[]){
    int thread_num = atoi(argv[1]);
    int m = atoi(argv[2]);
    int n = atoi(argv[3]);
    int k = atoi(argv[4]);

    // prepare matrix
    int* A = (int *)malloc(m * n * sizeof(int));
    int* B = (int *)malloc(n * k * sizeof(int));
    int* C = (int *)malloc(m * k * sizeof(int));

    srand(time(NULL));
    for (int i = 0; i < m * n; i++) {
        A[i] = rand() % 1000;
    }
    for (int i = 0; i < n * k; i++) {
        B[i] = rand() % 1000;
    }
}
```

首先，从命令行输入中分离出线程数和矩阵规模，并生成随机矩阵。

```
omp_set_num_threads(thread_num);
double start_time = omp_get_wtime();
matrix_multiply(A,B,C,m,n,k);
double end_time = omp_get_wtime();
double runtime = end_time - start_time;
printf("Matrix A: %d * %d, Matrix B: %d * %d, Threads: %d, Runtime: %.4f\n",m,n,n,k,thread_num,runtime);

printf("matrix A: \n");
```

```

seconds(n, m, n, n, k, time_start, time_end);

printf("matrix A: \n");
for(int i=0; i<m; i++){
    for(int j=0; j<n; j++){
        printf("%d ", A[i*n+j]);
    }
    printf("\n");
}

printf("matrix B: \n");
for(int i=0; i<n; i++){
    for(int j=0; j<k; j++){
        printf("%d ", A[i*k+j]);
    }
    printf("\n");
}

printf("matrix C: \n");
for(int i=0; i<m; i++){
    for(int j=0; j<k; j++){
        printf("%d ", A[i*k+j]);
    }
    printf("\n");
}

free(A);
free(B);
free(C);

```

随后，使用 `omp_set_num_threads` 设置线程数，使用 `omp_get_wtime()` 获得运行开始和运行结束的时间，调用自定义的 `matrix_multiply` 函数计算结果，并输出矩阵规模、进程数、运行时间、矩阵ABC。

```

void* matrix_multiply(int* A, int* B, int* C, int m, int n, int k){
    # pragma omp parallel for schedule(static)
    for(int i=0; i<m; i++){
        for(int j=0; j<k; j++){
            C[i * k + j] = 0;
            for(int a=0; a<n; a++){
                C[i*k+j] += A[i*n+a] * B[a*k+j];
            }
        }
    }
}

```

矩阵乘法函数和普通的矩阵乘法函数逻辑相似，区别是需要添加命令 `pragma omp parallel for`。

`schedule(static)` 表示当前的调度方式。OpenMP调度方式主要有：

1. 静态调度。将循环迭代均匀地分配给线程，是默认调度方式。
2. 动态调度。将循环迭代分成多个小任务，每次给每个线程分配一个任务。线程完成一个小任务后，会立刻获取下一个小任务。
3. GUIDED调度。GUIDED最初会给每个线程分配较大的任务，但是随着剩余迭代次数的减少，给每个线程分配的任务规模会减少，直到达到指定的最小大小。

实验结果

静态调度

线程数/矩阵规模	128	256	512	1024	2048
1	0.0097	0.0753	0.6495	6.6069	85.9793
2	0.0052	0.0416	0.3198	3.1177	34.7929
4	0.0077	0.0694	0.3537	3.1716	33.8524
8	0.0052	0.0532	0.3221	3.1321	34.5267
16	0.0060	0.0630	0.3539	3.4790	35.2227

动态调度

线程数/矩阵规模	128	256	512	1024	2048
1	0.0081	0.0892	0.6989	7.2413	79.2924
2	0.0052	0.0499	0.3516	3.2083	34.8673
4	0.0055	0.0589	0.3373	3.1676	33.6564
8	0.0057	0.0414	0.3312	3.1311	34.3589
16	0.0054	0.0497	0.3227	3.1546	34.2743

GUIDED调度

线程数/矩阵规模	128	256	512	1024	2048
1	0.0099	0.0704	0.6759	6.8429	83.0656
2	0.0051	0.0420	0.3350	3.1448	34.2221
4	0.0053	0.0494	0.3560	3.1627	35.1662
8	0.0066	0.0423	0.3365	3.6058	35.4448
16	0.0062	0.0430	0.3194	3.2647	36.3659

结果分析

在调度方式一定、矩阵规模一定的情况下，随着线程数增加，整体上运行时间呈下降趋势，但可能由于创建与合并线程的开销，运行时间并非严格下降。

在调度方式一定、线程数一定的情况下，随着矩阵规模增加，运行时间增加。

在矩阵规模一定、线程数一定的情况下，调度方式对运行时间没有明显的影响。可能是因为，本实验中所有任务的复杂度都类似，不存在会给某个线程分配复杂任务、导致其他线程需要等待这个线程执行完的情况，因此调度方式对性能没有明显影响。

部分矩阵打印结果

```
matrix A:
802 946 326 422 845 215 872 636 599 370 573 248 719 342 866 513 882 354 766 21
809 912 281 757 964 633 31 437 970 601 164 124 547 490 547 745 705 771 733 304
493 306 905 213 0 123 726 883 829 845 256 638 109 889 395 74 522 778 511 844 37
9 27 321 279 517 868 24 574 991 757 231 484 64 136 697 64 611 776 299 440 973 9
07 78 82 796 825 508 670 603 372 867 335 399 188 614 269 408 638 843 751 747 74
235 811 562 285 228 173 413 527 613 738 435 43 172 583 868 681 606 824 53 825
159 804 13 773 73 773
763 269 524 510 695 111 674 610 396 902 783 161 781 749 899 568 144 72 152 13 1
05 110 837 510 935 348 314 300 473 740 73 236 361 949 98 56 60 772 666 809 26 8
02 970 808 903 222 376 47 294 880 412 751 990 249 261 277 949 927 929 422 667 3
54 658 28 303 109 437 364 881 455 173 908 257 495 716 160 717 92 560 363 325 97
2 114 315 574 727 945 523 655 874 946 322 229 956 703 532 65 492 248 299 947 42
1 559 557 917 275 69 986 719 629 350 44 602 816 712 176 544 657 51 199 883 997
873 464 306 928 997 371
```

实验感想

在本次实验中，我学会了使用OpenMP API进行并行矩阵乘法运算，了解了OpenMP并行的不同调度方式。在本次实验中，我没有遇到复杂的困难。